

Bài tập về nhà Tuần 13 - 19/05/2026

Môn Lập trình mạng

Họ và tên: Trần Dương An

MSSV: 20235256

Bài 1: Chương trình *File Server*

❖ Đề bài:

Sử dụng kỹ thuật đa tiến trình, lập trình ứng dụng file server đơn giản như sau:

+) Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi “**OK N\r\n**” nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự **\r\n**, kết thúc danh sách bởi ký tự **\r\n\r\n**.

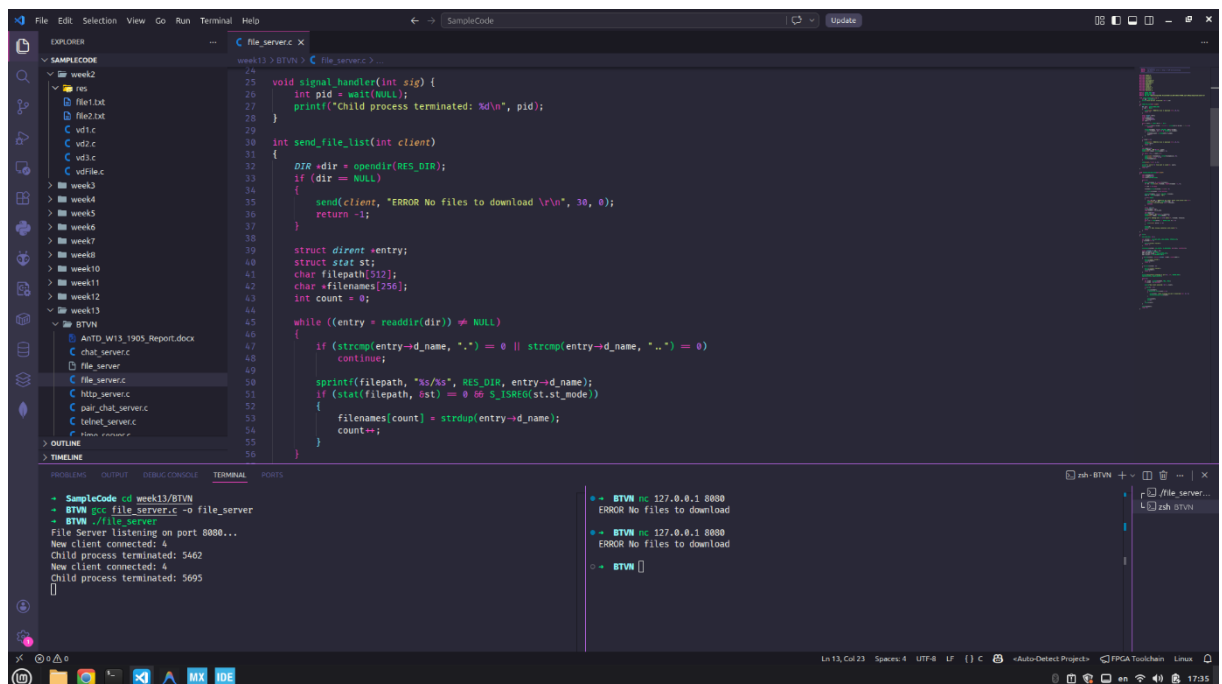
+) Nếu không có file nào trong thư mục thì server gửi chuỗi “**ERROR No files to download\r\n**” rồi đóng kết nối.

+) Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi “**OK N\r\n**” trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

❖ Link mã nguồn: [File Server](#)

❖ Demo chương trình:

1. Trường hợp không có file trong thư mục



The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project structure with folders for weeks 1 through 13, and a folder named BTVN. The code editor shows the implementation of the File Server program. The terminal shows the output of the program, including the command to run the file server and the resulting output.

```
void signal_handler(int sig) {
    int pid = wait(NULL);
    printf("Child process terminated: %d\n", pid);
}

int send_file_list(int client)
{
    DIR *dir = opendir(RES_DIR);
    if (dir == NULL)
    {
        send(client, "ERROR No files to download \r\n", 30, 0);
        return -1;
    }

    struct dirent *entry;
    struct stat st;
    char filepath[512];
    char *filenames[256];
    int count = 0;

    while ((entry = readdir(dir)) != NULL)
    {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
            continue;
        sprintf(filepath, "%s/%s", RES_DIR, entry->d_name);
        if (stat(filepath, &st) == 0 && S_ISREG(st.st_mode))
        {
            filenames[count] = strdup(entry->d_name);
            count++;
        }
    }
}
```

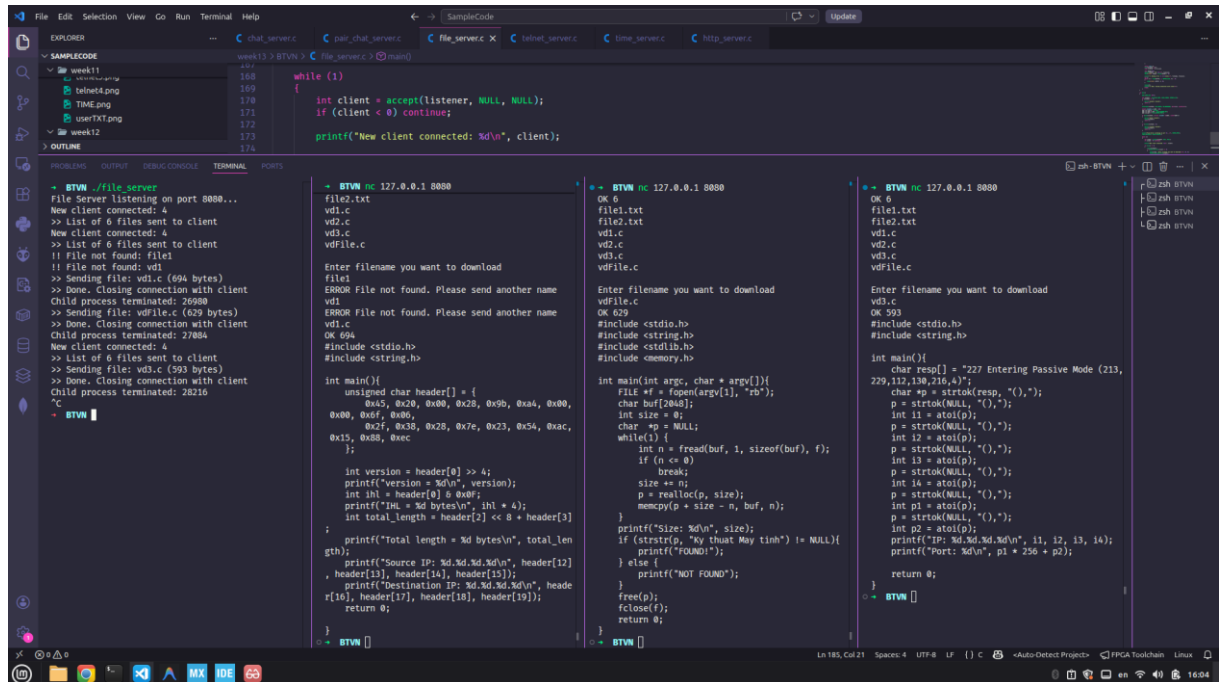
```
SampleCode cd week13/BTVN
BTVN gcc file_server.c -o file_server
BTVN ./file_server
File Server listening on port 8080...
New client connected: 4
Child process terminated: 5462
New client connected: 4
Child process terminated: 5695
[]
```

```
BTVN 127.0.0.1 8080
ERROR No files to download
BTVN 127.0.0.1 8080
ERROR No files to download
BTVN []
```

Hình 1 – Demo 1 File Server

Giải thích:

- Biên dịch và chạy chương trình **file_server**. Kết quả: *File Server is listening on port 8080...*
 - Giả lập client bằng netcat bằng lệnh `nc 127.0.0.1 8080`, sau khi kết nối tới server thành công thì nhận được thông báo *ERROR No files to download*. Đồng thời server cũng đóng kết nối và ghi lại log.
2. Trường hợp có file



Hình 2 – Demo 2 File Server

Giải thích:

- Biên dịch và chạy chương trình **file_server**. Kết quả: *File Server is listening on port 8080...*
- Giả lập nhiều client bằng netcat bằng lệnh `nc 127.0.0.1 8080`, sau khi kết nối tới server thành công thì nhận được thông báo *OK 6* và danh sách gồm tên 6 file. Sau đó server cũng gửi thông báo *Enter filename you want to download*.
- Các client nhập tên file cần download, nếu nhập đúng 1 trong 6 file trên thì download thành công, ngược lại nhận được thông báo *ERROR File not found. Please send another name*.

Bài 2: Chương trình Pair Chat Server❖ **Đề bài:**

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

+) Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.

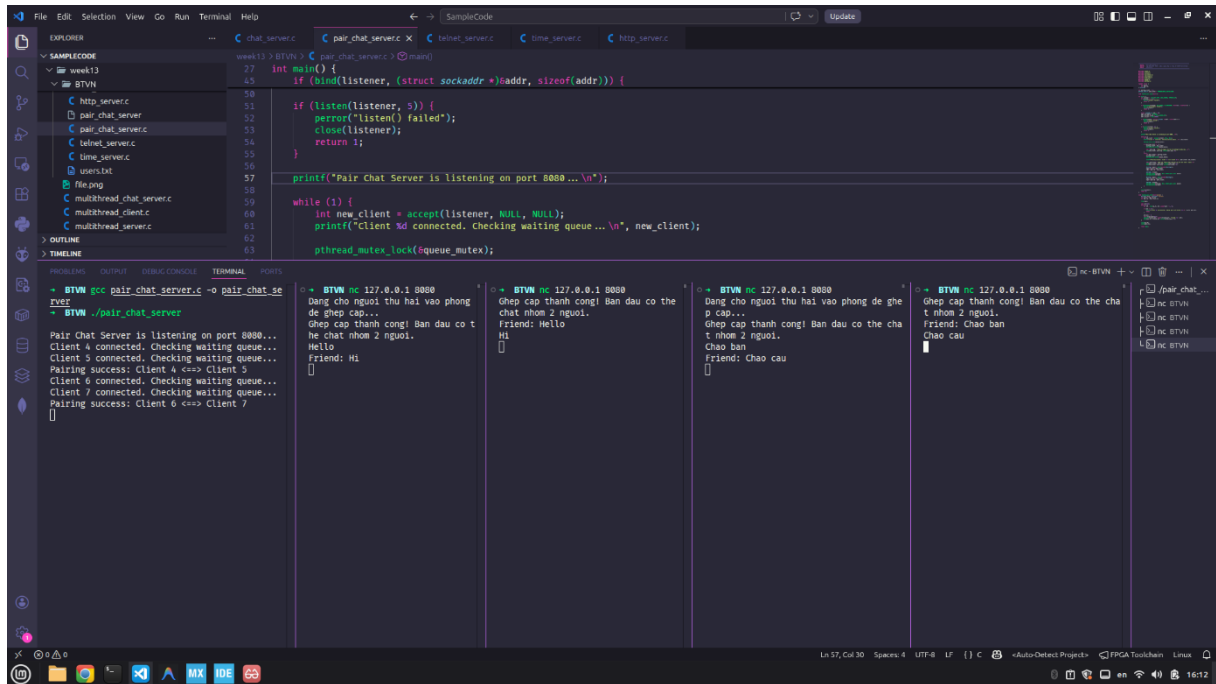
+) Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.

+) Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

❖ *Link mã nguồn:* [Pair Chat Server](#)

❖ *Demo chương trình:*

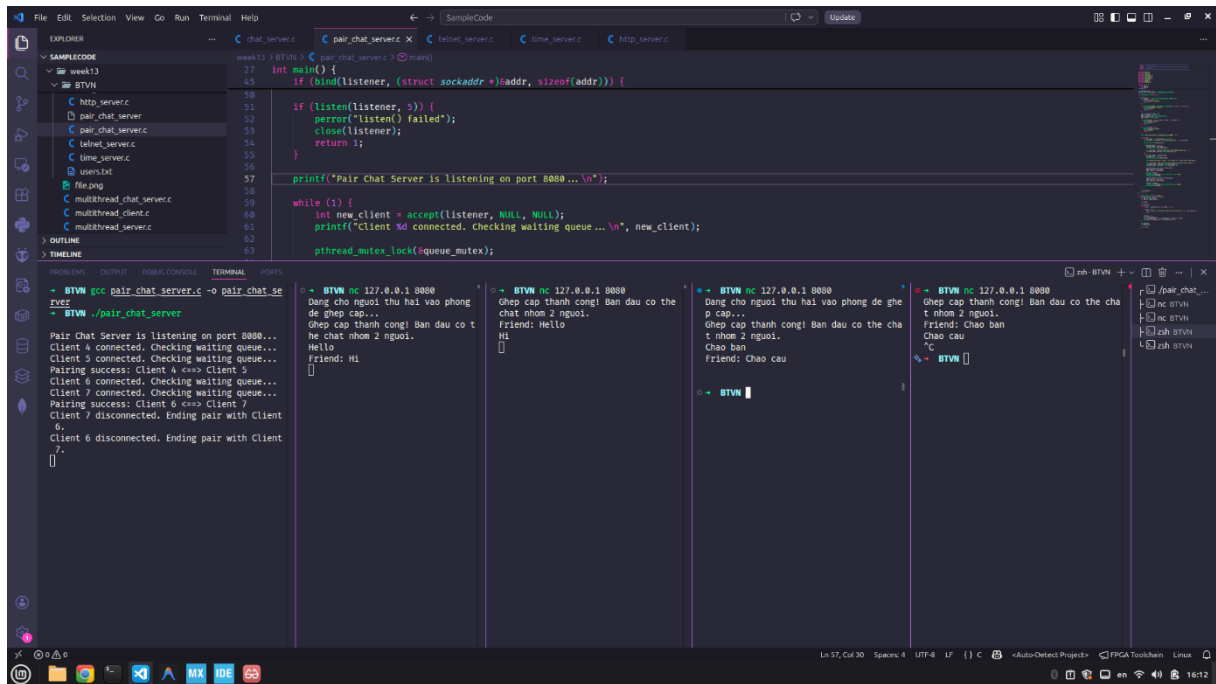
Ta sẽ minh họa trên 5 Terminal từ trái qua phải lần lượt là Pair Chat Server, Client 4, Client 5, Client 6, Client 7 (Xem ảnh dưới).



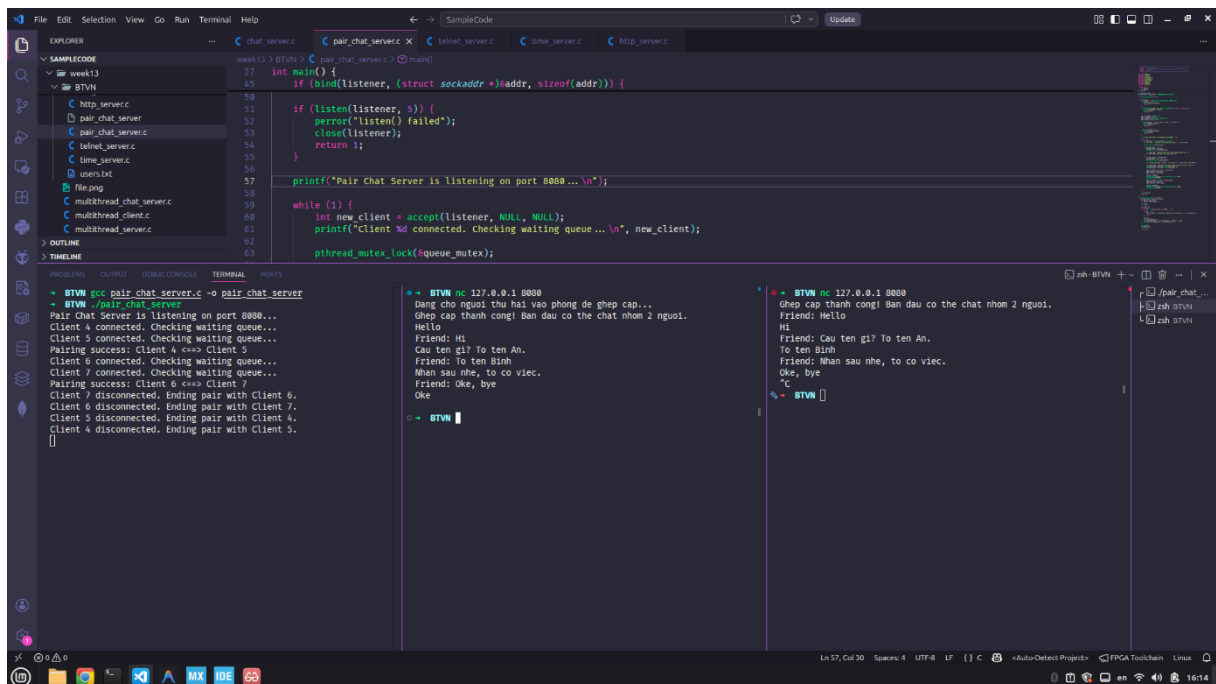
Hình 3 – Demo 1 Pair Chat Server

Giải thích:

- Biên dịch và chạy chương trình **pair_chat_server**. Kết quả: *Pair Chat Server is listening on port 8080...*
- Giả lập các client bằng netcat bằng lệnh `nc 127.0.0.1 8080`. Lần lượt cho Client 4, 5, 6, 7 kết nối tới server. Kết quả là Client 4 nhận được thông báo chờ ghép cặp, Client 5 sau khi kết nối được ghép cặp với Client 4. Tương tự với Client 6 và 7.
- Client 4 nhắn thì Client 5 nhận được và ngược lại. Client 6 nhắn thì Client 7 nhận được và ngược lại.
- Client 7 ngắt kết nối. Server cũng ngắt kết nối luôn với Client 6 (Xem log trên Server ở hình 4).
- Ta giữ lại Client 4 và 5 trao đổi tin nhắn với nhau qua lại vẫn bình thường (Xem hình 5)
- Client 5 ngắt kết nối. Server cũng ngắt kết nối luôn với Client 4, Client 4 có gửi thêm Ok nhưng 5 không nhận vì đã ngắt kết nối rồi. (Xem hình 5).



Hình 4 – Demo 2 Pair Chat Server



Hình 5 – Demo 3 Pair Chat Server

Bài 3: Chương trình *Chat Server*

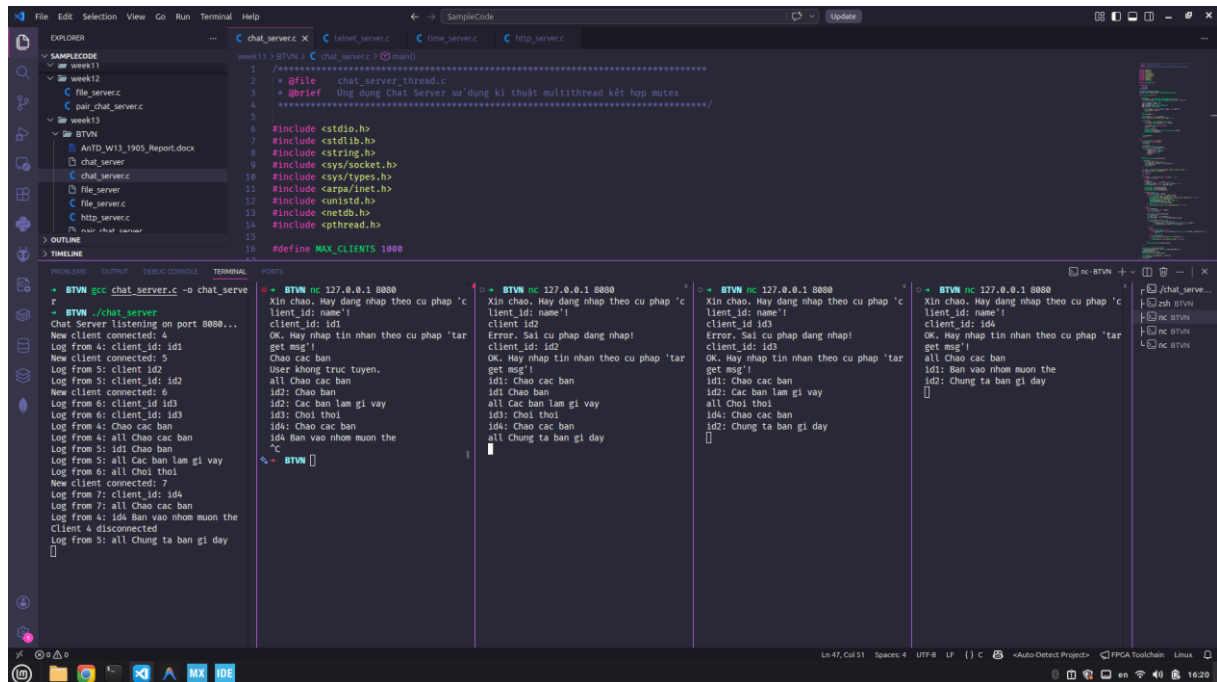
❖ Đề bài:

Lập trình ứng dụng *chat_server* (slide 173) sử dụng kỹ thuật *multithread*.

❖ Link mã nguồn: [Chat Server](#)

❖ Demo chương trình:

Ta sẽ minh họa trên 5 Terminal từ trái qua phải lần lượt là Chat Server, Client id1, Client id2, Client id3 và Client id4 (Xem ảnh dưới).



Hình 6 – Demo 1 Chat Server

Giải thích:

- Biên dịch và chạy chương trình **chat_server**. Kết quả: *Chat Server listening on port 8080...*
- Giả lập các client bằng netcat bằng lệnh **nc 127.0.0.1 8080**, sau khi kết nối tới server thành công thì đều nhận được thông báo *Xin chào. Hay dang nhap theo cu phap 'client_id: client_name'!*. Đồng thời server cũng ghi lại log.
- Các Client lần lượt đăng nhập, nếu sai cú pháp server báo lỗi và yêu cầu đăng nhập lại, thành công thì server thông báo cú pháp gửi tin nhắn.
- Các Client nhắn tin với nhau nếu muốn gửi cho mọi người thì nhấn 'all xxxxx' còn cụ thể thì nhấn kèm client_id, ví dụ 'id1Chao ban'.
- Các Client nhận được tin nhắn đúng như mong đợi.

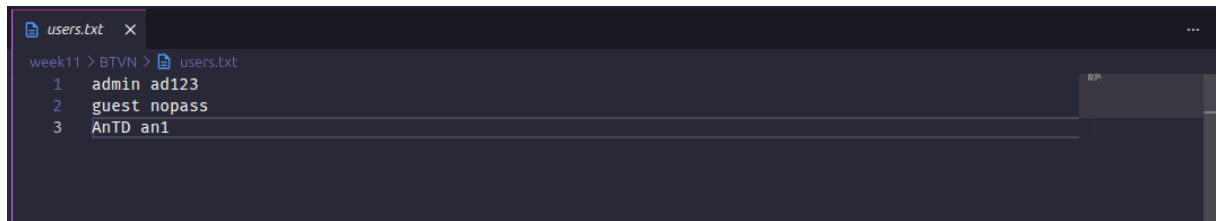
Bài 4:

❖ *Đề bài:*

Lập trình lại ứng dụng **telnet_server** (slide 174) sử dụng kỹ thuật **multithread**.

❖ *Link mã nguồn:* [Telnet Server](#)

❖ *Demo chương trình:*

1. Nội dung file users.txt


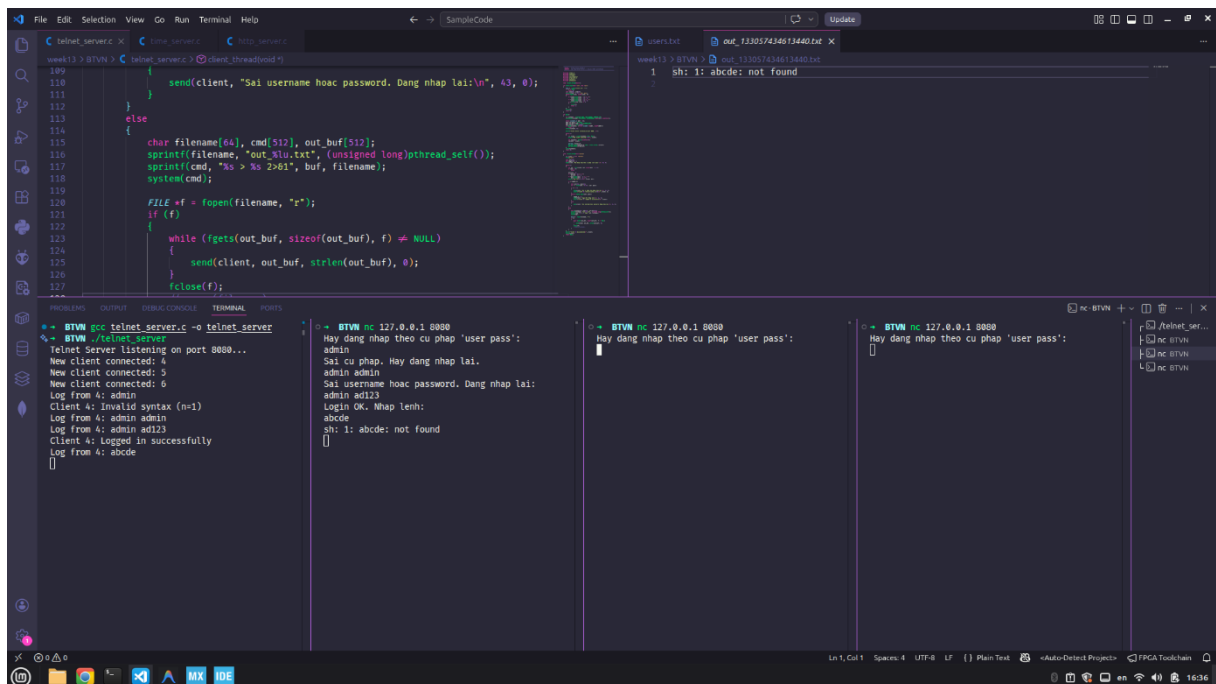
```

week11 > BTVN > users.txt
1  admin ad123
2  guest nopass
3  AnTD an1

```

Hình 7 – Nội dung file users.txt

2. Ta sẽ minh họa trên 4 Terminal lần lượt từ trái qua phải như sau: Telnet Server, Client admin ad123 (Admin), Client guest nopass (Khách) Client AnTD an1 (An), (Xem Hình 8)



```

// telnet_server.c
189
190
191     send(client, "Sai username hoặc password. Dang nhap lai:\n", 43, 0);
192 }
193 else
194 {
195     char filename[64], cmd[512], out_buf[512];
196     sprintf(filename, "out_%lu.txt", (unsigned long)pthread_self());
197     sprintf(cmd, "ls > %s 2>&1", buf, filename);
198     system(cmd);
199
200     FILE *f = fopen(filename, "r");
201     if (f)
202     {
203         while (fgets(out_buf, sizeof(out_buf), f) != NULL)
204         {
205             send(client, out_buf, strlen(out_buf), 0);
206         }
207         fclose(f);
208     }
209 }

```

```

week13 > BTVN > out_133057434613440.txt
1  sh: 1: abcde: not found

```

```

week13 > BTVN > telnet_server.c
1  admin ad123
2  guest nopass
3  AnTD an1

```

```

week13 > BTVN > telnet_server.c
1  admin ad123
2  guest nopass
3  AnTD an1

```

```

week13 > BTVN > telnet_server.c
1  admin ad123
2  guest nopass
3  AnTD an1

```

Hình 8 – Demo 1 Telnet Server

Giải thích:

- Biên dịch và chạy chương trình **telnet_server**. Kết quả: *Telnet Server listening on port 8080...*
- Giả lập các client bằng netcat bằng lệnh `nc 127.0.0.1 8080`, sau khi kết nối tới server thành công thì đều nhận được thông báo *Hay dang nhap theo cu phap 'user pass':*
- Admin lần lượt nhập sai cú pháp cũng như sai thông tin đăng nhập, server báo về lỗi tùy từng trường hợp như *Sai cu phap. Hay dang nhap lai* hoặc *Sai username hoặc password. Dang nhap lai*. Sau đó, Admin nhập đúng user pass thì đăng nhập thành công và server báo về *Login OK. Hay nhap lenh:*
- Admin nhập lệnh *abcde* (đây là một lệnh không có thật), server nhận lệnh, thực thi không được, ghi vào file **out_133057434613440.txt** nội dung *sh: 1: abcde: not found* (Xem hình 8 – góc trên phải), và trả về cho Admin nội dung file này.

- Khách đăng nhập, gửi lệnh `pwd`, server nhận được, thực thi, kết quả ghi vào file **out_133057426220736.txt** (nội dung ở hình 9 – góc trên phải), và trả về cho Khách nội dung file này.
- An đăng nhập, gửi lệnh `ls -l`, server nhận được, thực thi, kết quả ghi vào file **out_133057348630208.txt** (nội dung ở hình 10 – góc trên phải), và trả về cho An nội dung file này.
- Admin, Khách lần lượt ngắt kết nối, server biết và ghi lại log cho hành động này. (Xem hình 10 – góc dưới bên trái)

The screenshot shows a Visual Studio Code editor with a C program for a Telnet server. The code is located in `SampleCode` and includes a `client_thread` function that handles client requests. The terminal output shows the server listening on port 8080 and handling several clients. The logs show the server receiving commands like `admin`, `admin admin`, `admin ad123`, `guest nopass`, and `pwd`. The logs also show the server sending responses back to the clients.

Hình 9 – Demo 2 Telnet Server

The screenshot shows the same Visual Studio Code editor with the Telnet server code. The terminal output shows the server handling more clients. The logs show the server receiving commands like `admin`, `admin admin`, `admin ad123`, `guest nopass`, and `pwd`. The logs also show the server sending responses back to the clients. The logs also show the server receiving commands like `ls -l` and `total 2342`.

Hình 10 – Demo 3 Telnet Server

Bài 5:

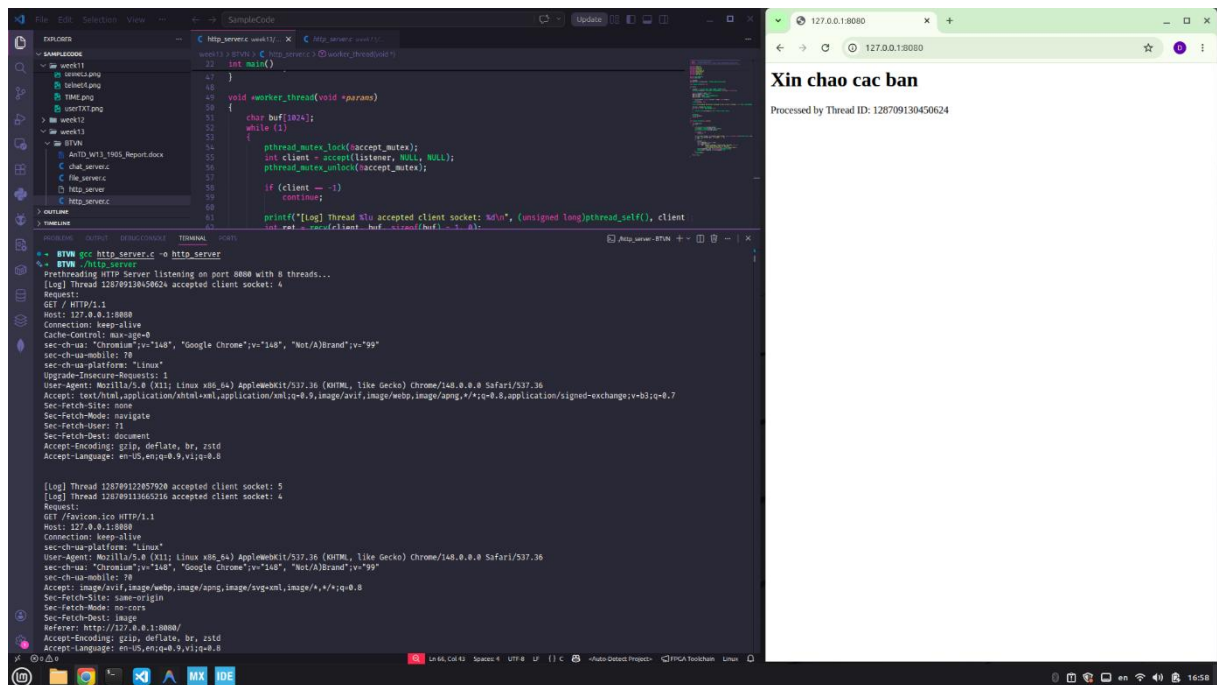
◆ *Đề bài:*

Lập trình lại ứng dụng *http_server* (slide 154) sử dụng kỹ thuật *prethreading*.

- ❖ *Link mã nguồn:* [HTTP Server](#)
- ❖ *Demo chương trình:*

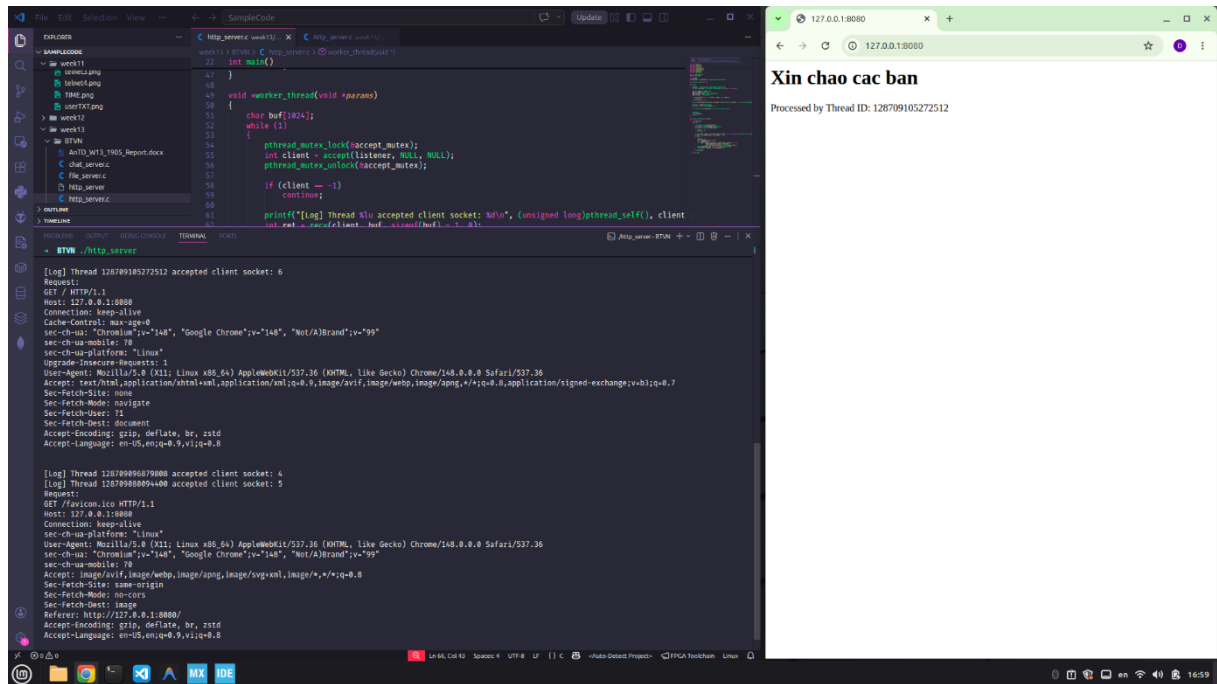
Giải thích:

- Biên dịch và chạy chương trình **http_server**. Kết quả: *Prethreading HTTP Server listening on port 8080 with 8 threads...*
- Mở trình duyệt và nhập địa chỉ 127.0.0.1:8080, thì nhận được trang web có nội dung như Hình 11. Đồng thời phía server cũng ghi lại log cho hành động này. Lưu ý lúc này client được phục vụ bởi Thread24.

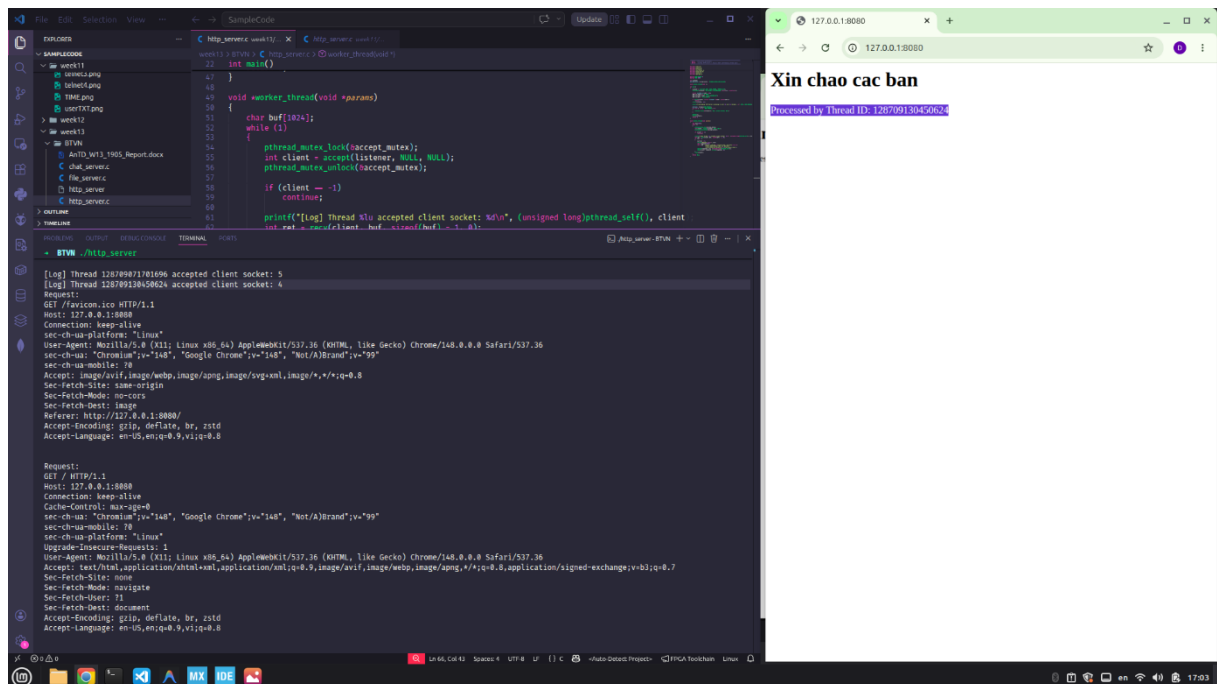


Hình 11 – Demo 1 HTTP Server

- Refresh trang này (tương đương với việc có 1 client mới kết nối) thì cũng nhận được trang web có nội dung như Hình 12 (tương tự Hình 11). Đồng thời phía server cũng ghi lại log cho hành động này. Lưu ý lúc này client được phục vụ bởi Thread12 (Khác với Thread24 ở trên).
- Refresh thêm nhiều lần nữa thì Client lúc ấy lại được phục vụ bởi Thread24 (Xem hình 13), Thread này chính là Thread ở bước đầu tiên.



Hình 12 – Demo 2 HTTP Server



Hình 13 – Demo 3 HTTP Server

Bài 6:

❖ Đề bài:

Lập trình ứng dụng ***time_server*** (slide 204) sử dụng kỹ thuật ***multithread***.

❖ Link mã nguồn: [Time Server](#)

❖ Demo chương trình:

Ta sẽ minh họa trên 3 Terminal từ trái qua phải lần lượt là Time Server, Client A, Client B (Xem Hình 13).

```

// time_server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <pthread.h>
#include <time.h>

#define PORT 8080
#define BUFFER_SIZE 256

void *client_thread(void *params)
{
    int client = *(int *)params;
    free(params);

    char *welcome_msg = "Chao ban! Day la Time Server.\n";
    "Hay nhap lenh theo cu phap: GET_TIME [format]\n";
    "Cac dinh dang ho tro:\n";
    "- dd/mm/yyyy\n";
    "- dd/mm/yy\n";
    "- mm/dd/yyyy\n";
    "- mm/dd/yy\n";

    send(client, welcome_msg, strlen(welcome_msg), 0);

    char buf[BUFFER_SIZE];
    while (1)
    {
        int len = recv(client, buf, sizeof(buf) - 1, 0);
        if (len < 0)
            break;

        buf[len] = 0;
        if (buf[len - 1] == '\n')
            buf[len - 1] = 0;
        if (buf[strlen(buf) - 1] == '\r')
            buf[strlen(buf) - 1] = 0;
        printf("Log from %d: %s\n", client, buf);

        if (strcmp(buf, "GET_TIME") == 0)
        {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char date[20];
            strftime(date, sizeof(date), "%d/%m/%Y", t);
            send(client, date, strlen(date), 0);
        }
        else if (strcmp(buf, "GET_TIME dd/mm/yyyy") == 0)
        {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char date[20];
            strftime(date, sizeof(date), "%d/%m/%Y", t);
            send(client, date, strlen(date), 0);
        }
        else if (strcmp(buf, "GET_TIME dd/mm/yy") == 0)
        {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char date[20];
            strftime(date, sizeof(date), "%d/%m/%y", t);
            send(client, date, strlen(date), 0);
        }
        else if (strcmp(buf, "GET_TIME mm/dd/yyyy") == 0)
        {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char date[20];
            strftime(date, sizeof(date), "%m/%d/%Y", t);
            send(client, date, strlen(date), 0);
        }
        else if (strcmp(buf, "GET_TIME mm/dd/yy") == 0)
        {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char date[20];
            strftime(date, sizeof(date), "%m/%d/%y", t);
            send(client, date, strlen(date), 0);
        }
        else
        {
            send(client, "Error: Sai cu phap. Yeu cau: GET_TIME [format].\n", 40, 0);
        }
    }
}

int main()
{
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int n = 1;
    socklen_t addrlen = sizeof(address);

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        printf("socket failed\n");
        return -1;
    }

    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, opt, sizeof(int)) < 0)
    {
        printf("setsockopt SO_REUSEADDR failed\n");
        return -1;
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
    {
        printf("bind failed\n");
        return -1;
    }

    if (listen(server_fd, 5) < 0)
    {
        printf("listen failed\n");
        return -1;
    }

    pthread_t t1;
    while (n)
    {
        new_socket = accept(server_fd, (struct sockaddr *)&address, &addrlen);
        pthread_create(&t1, NULL, client_thread, (void *)&new_socket);
        n++;
    }

    return 0;
}

```

Terminal 1 (Time Server):

```

$ gcc time_server.c -o time_server
$ ./time_server
Time Server listening on port 8080...
New client connected: 4
New client connected: 5
Log from 4: GET
Log from 4: GET_TIME dd/mm/yyyy
Log from 4: GET_TIME dd/mm/yy
Log from 5: GET_TIME mm/dd/yyyy
Log from 5: GET_TIME mm/dd/yy
Log from 5: GET_TIME yyyy/mm/dd
Client 4 disconnected
Client 4 disconnected

```

Terminal 2 (Client A):

```

$ nc 127.0.0.1 8080
Chao ban! Day la Time Server.
Hay nhap lenh theo cu phap: GET_TIME [format]
Cac dinh dang ho tro:
- dd/mm/yyyy
- dd/mm/yy
- mm/dd/yyyy
- mm/dd/yy
GET
Error: Sai cu phap. Yeu cau: GET_TIME [format]
GET_TIME
Error: Sai cu phap. Yeu cau: GET_TIME [format]
GET_TIME dd/mm/yyyy
24/05/2026
GET_TIME dd/mm/yy
24/05/26

```

Terminal 3 (Client B):

```

$ nc 127.0.0.1 8080
Chao ban! Day la Time Server.
Hay nhap lenh theo cu phap: GET_TIME [format]
Cac dinh dang ho tro:
- dd/mm/yyyy
- dd/mm/yy
- mm/dd/yyyy
- mm/dd/yy
GET_TIME mm/dd/yyyy
05/24/2026
GET_TIME mm/dd/yy
05/24/26
GET_TIME yyyy/mm/dd
Error: Dinh dang thoi gian khong duoc ho tro.

```

Hình 14 – Demo Time Server

Giải thích:

- Biên dịch và chạy chương trình **time_server**. Kết quả: *Time Server listening on port 8080...*
- Giả lập các client bằng netcat bằng lệnh **nc 127.0.0.1 8080**, sau khi kết nối tới server thành công thì đều nhận được thông báo

Chao ban! Day la Time Server.

Hay nhap lenh theo cu phap: GET_TIME [format]

Cac dinh dang ho tro:

- dd/mm/yyyy

- dd/mm/yy

- mm/dd/yyyy

- mm/dd/yy

Đồng thời server cũng ghi lại log.

- Client A lần lượt nhập lệnh **GET** và **GET_TIME**, là các lệnh sai nên server thông báo *Error: Sai cu phap. Yeu cau: GET_TIME [format]*. Sau đó, A nhập các lệnh đúng cú pháp và server trả về kết quả tương ứng.
- Client B nhập các lệnh đúng cú pháp và server trả về kết quả tương ứng. Sau đó, B nhập lệnh **GET_TIME m/d/y**, là lệnh sai định dạng thời gian nên server thông báo *Error: Dinh dang thoi gian khong duoc ho tro.*
- Client A và B ngắt kết nối, server ghi lại log.